THE UNIVERSITY OF MELBOURNE

COMP90038 Algorithms and Complexity

Lecture 5: Brute Force Methods

Olya Ohrimenko
(with thanks to Harald Søndergaard)

Announcements

- Weekly quizzes reminder
- Python code on LMS
- Tutorials, Ed and consultation hours

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

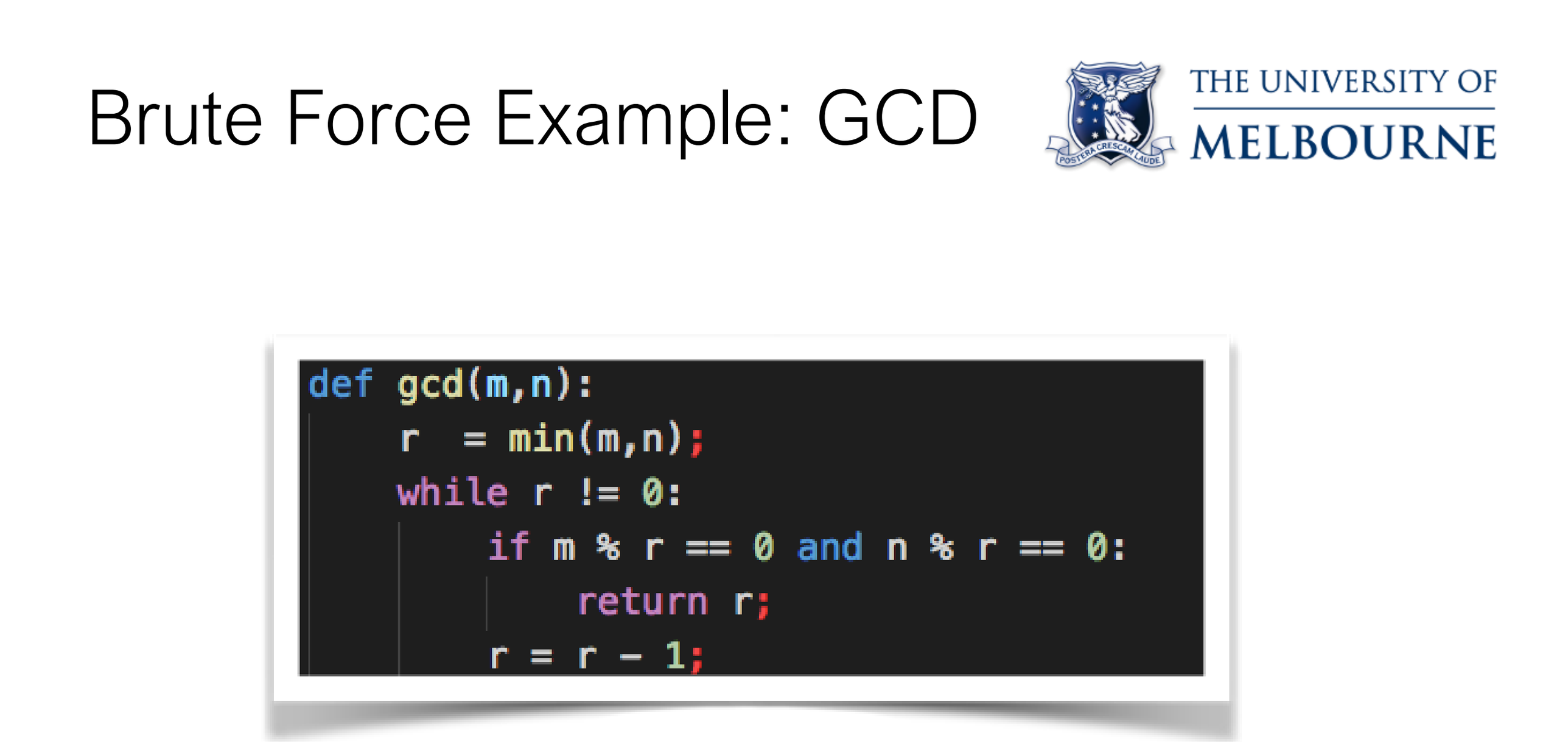
Brute Force Algorithms

- Straightforward problem solving approach, usually based directly on the problem's statement.

- Exhaustive search for solutions is a prime example.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Brute Force Example: GCD



```
def gcd(m,n):
    r  = min(m,n);
    while r != 0:
        if m % r == 0 and n % r == 0:
            return r;
        r = r - 1;
```


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Brute Force Algorithms

- Straightforward problem solving approach, usually based directly on the problem's statement.
- Exhaustive search for solutions is a prime example.
- Today's algorithms
  - Selection sort
  - String matching
  - Closest pair
  - Exhaustive search for combinatorial solutions
  - Graph traversal

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Selection Sort

```
function SelSort(A[·], n)
    for i ← 0 to n − 2 do
        min ← i
        for j ← i + 1 to n − 1 do
            if A[j] < A[min] then
                min ← j
        t ← A[i]                    // swap A[i] and A[min]
        A[i] ← A[min]
        A[min] ← t
```

Time Complexity: $\Theta(n^2)$

We will soon meet better sorting algorithms

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Properties of Sorting Algorithms

THE UNIVERSITY OF MELBOURNE

- A Sorting algorithm is:
  - **in-place** if it does not require additional memory except, perhaps, for a few units of memory
  - **input-insensitive** if its running time is fairly independent of input properties other than size
  - **stable** if it preserves the relative order of elements with identical keys

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Card images from wikipedia

Which properties hold?



THE UNIVERSITY OF MELBOURNE

```
function SelSort(A[·], n)
    for i ← 0 to n − 2 do
        min ← i
        for j ← i + 1 to n − 1 do
            if A[j] < A[min] then
                min ← j
        t ← A[i]
        A[i] ← A[min]
        A[min] ← t
```


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Properties of Selection Sort



THE UNIVERSITY OF MELBOURNE

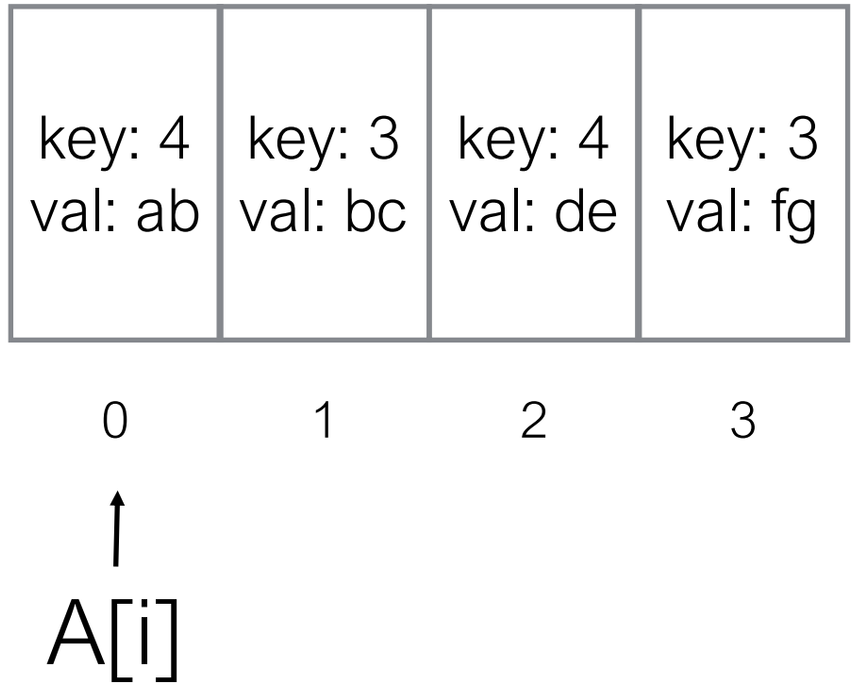
- While running time is quadratic, selection sort makes only about ***n* exchanges**.

- So: **selection sort is a good algorithm for sorting small collections of large records.**

- In-place? yes
- Input-insensitive? yes
- Stable? ?

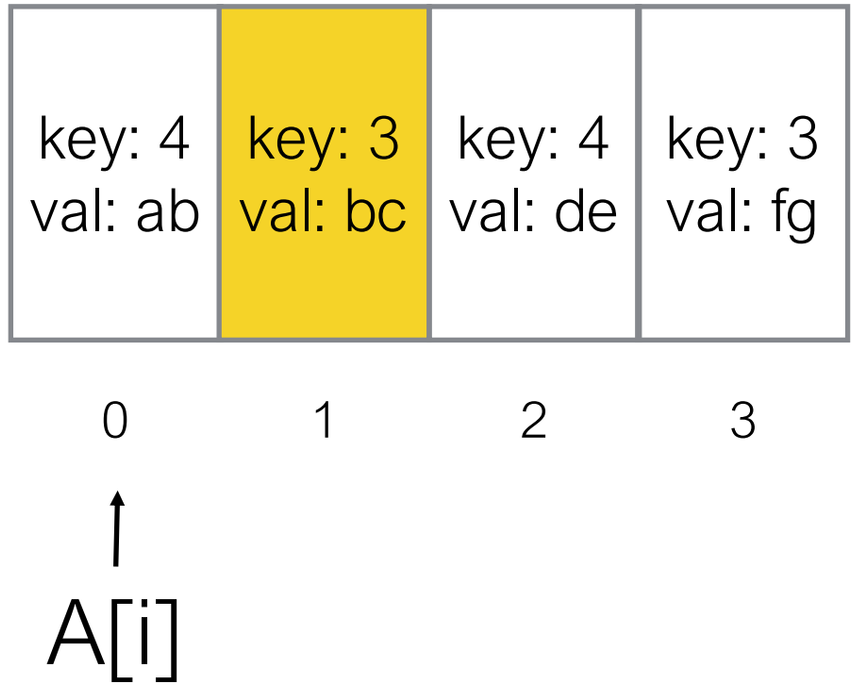
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Selection Sort Stability



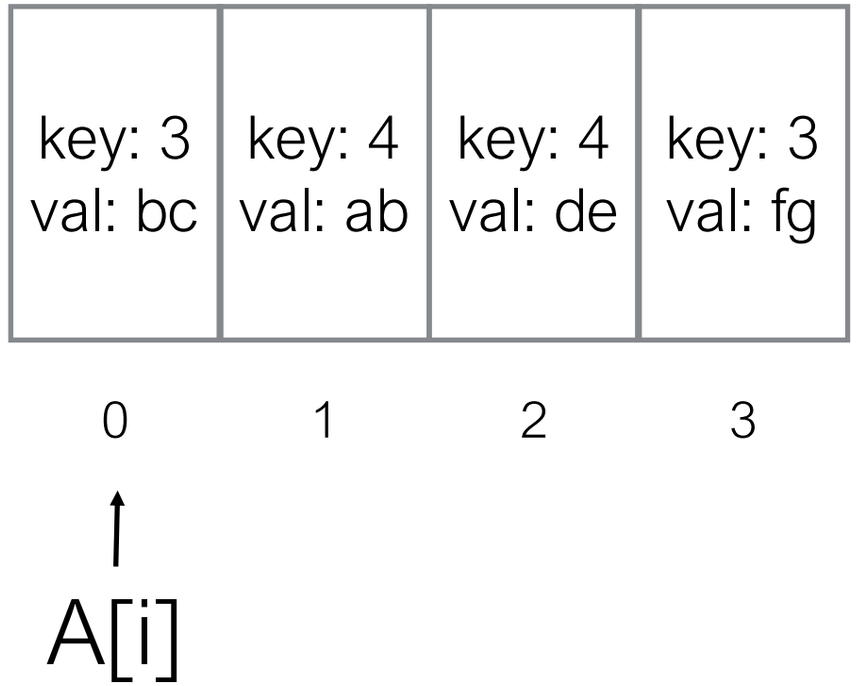
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Selection Sort Stability



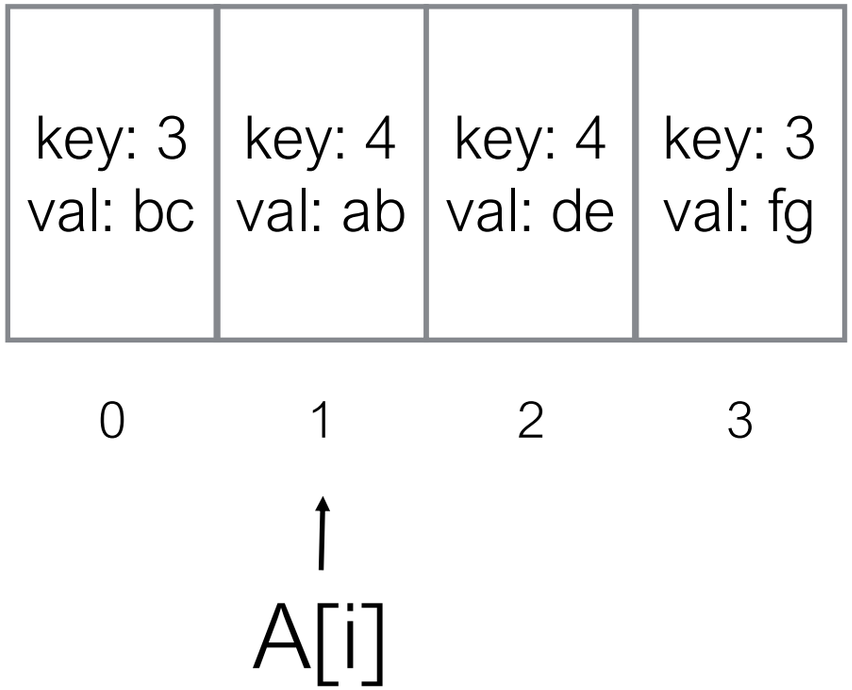
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Selection Sort Stability



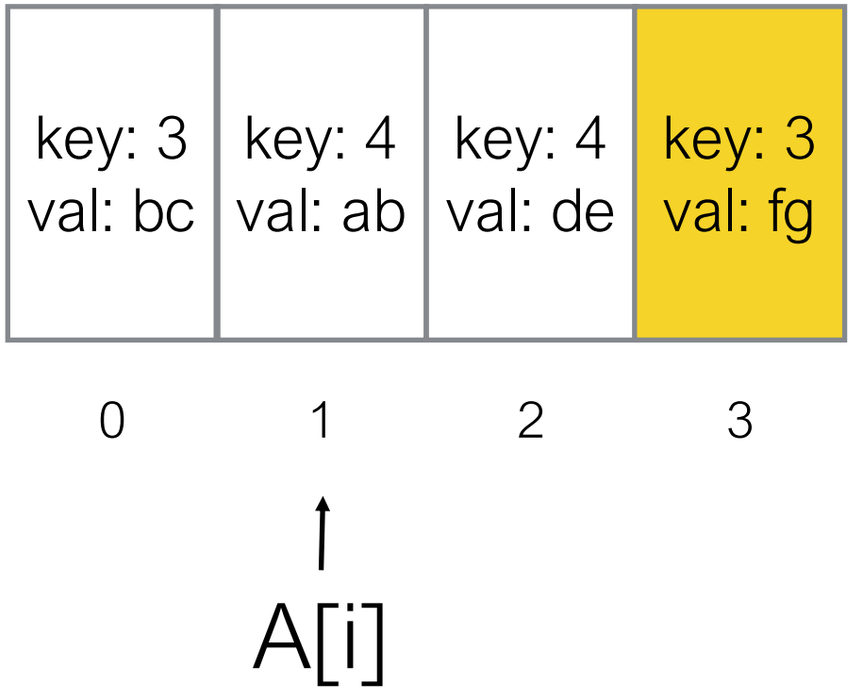
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Selection Sort Stability



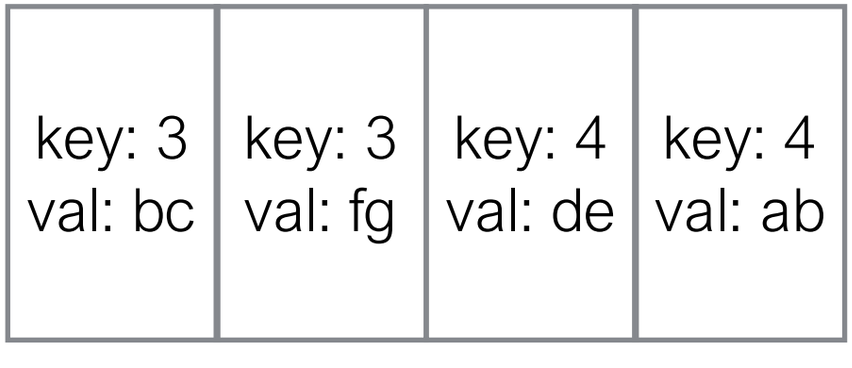
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Selection Sort Stability



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Selection Sort Stability



the relative order
of the two “4” records
has changed!

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Properties of Selection Sort



THE UNIVERSITY OF MELBOURNE

- While running time is quadratic, selection sort makes only about ***n* exchanges**.

- So: **selection sort is a good algorithm for sorting small collections of large records.**

- In-place? yes
- Input-insensitive? yes
- Stable? no

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Brute Force String Matching

- **Pattern** *p*: a string of *m* characters to search **for**
- **Text** *t*: a long string of *n* characters to search **in**



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Brute Force String Matching



THE UNIVERSITY OF MELBOURNE

- **Pattern** *p*: a string of *m* characters to search **for**
- **Text** *t*: a long string of *n* characters to search **in**
- We use *i* to run through the text and *j* to run through the pattern

```
for i ← 0 to n − m do
    j ← 0
    while j < m and p[j] = t[i + j] do
        j ← j + 1
    if j = m then
        return i
return −1
```

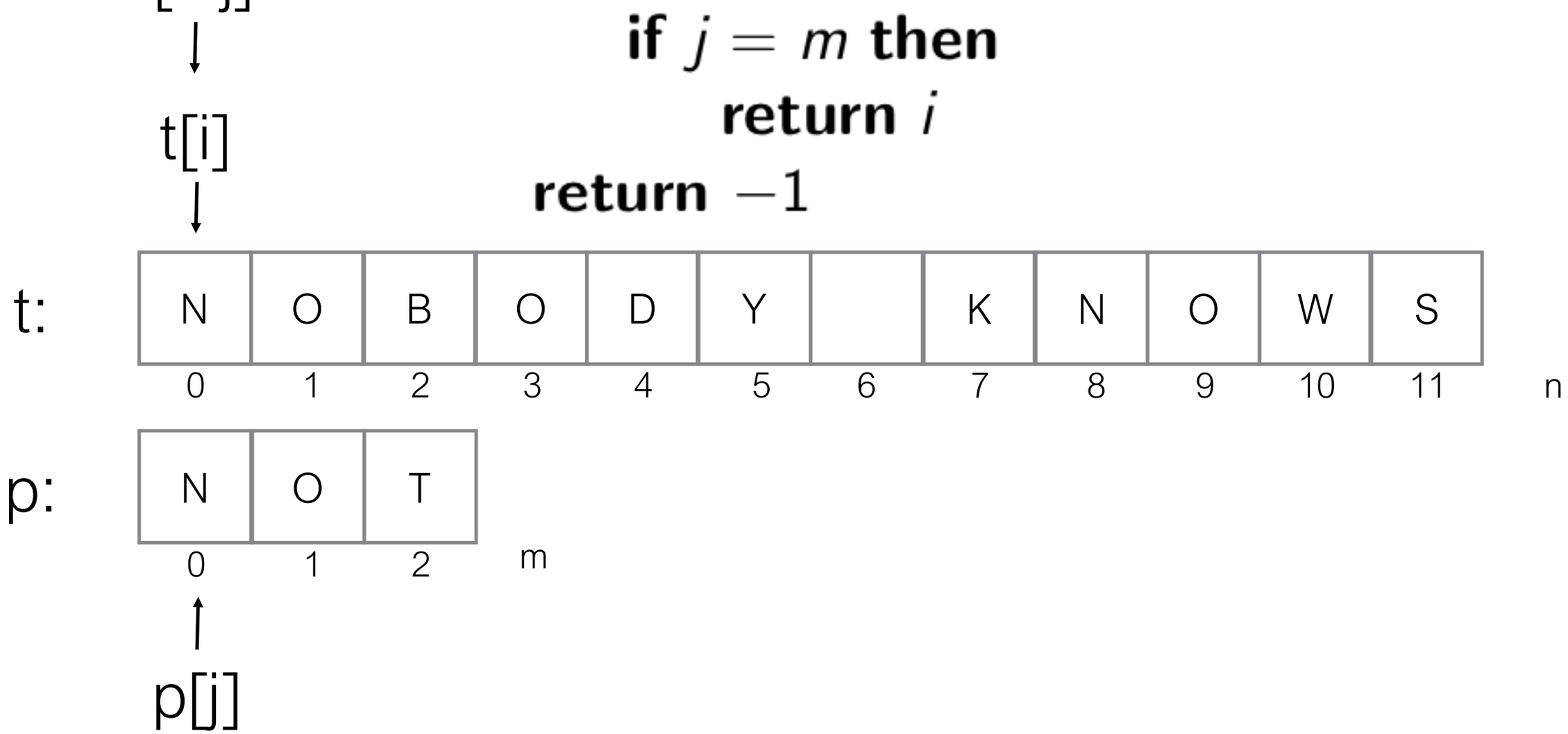

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Brute Force String Matching



THE UNIVERSITY OF MELBOURNE

```
for i ← 0 to n − m do
    j ← 0
    while j < m and p[j] = t[i + j] do
        j ← j + 1
    if j = m then
        return i
return −1
```



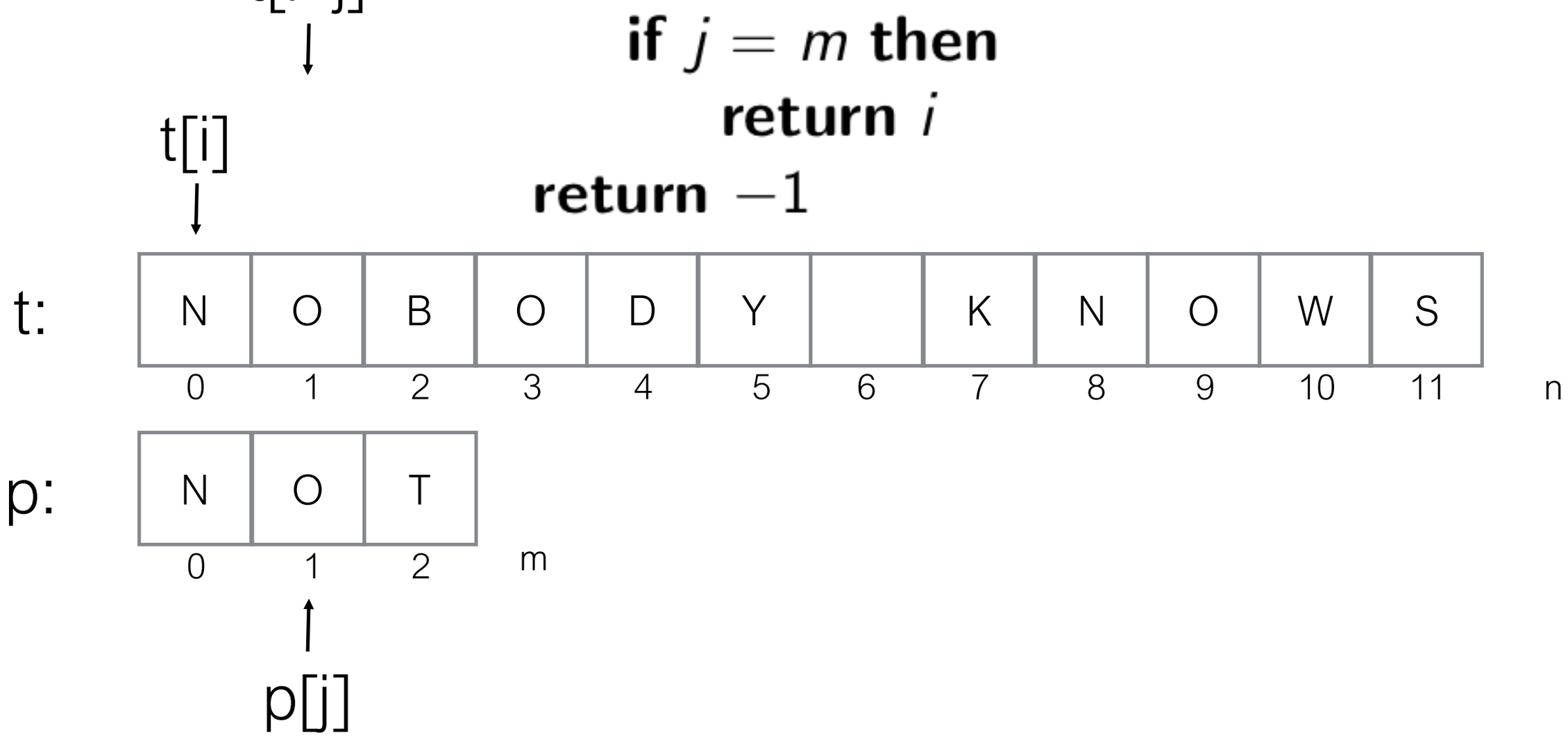
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Brute Force String Matching



THE UNIVERSITY OF MELBOURNE

```
for i ← 0 to n − m do
    j ← 0
    while j < m and p[j] = t[i + j] do
        j ← j + 1
    if j = m then
        return i
return −1
```



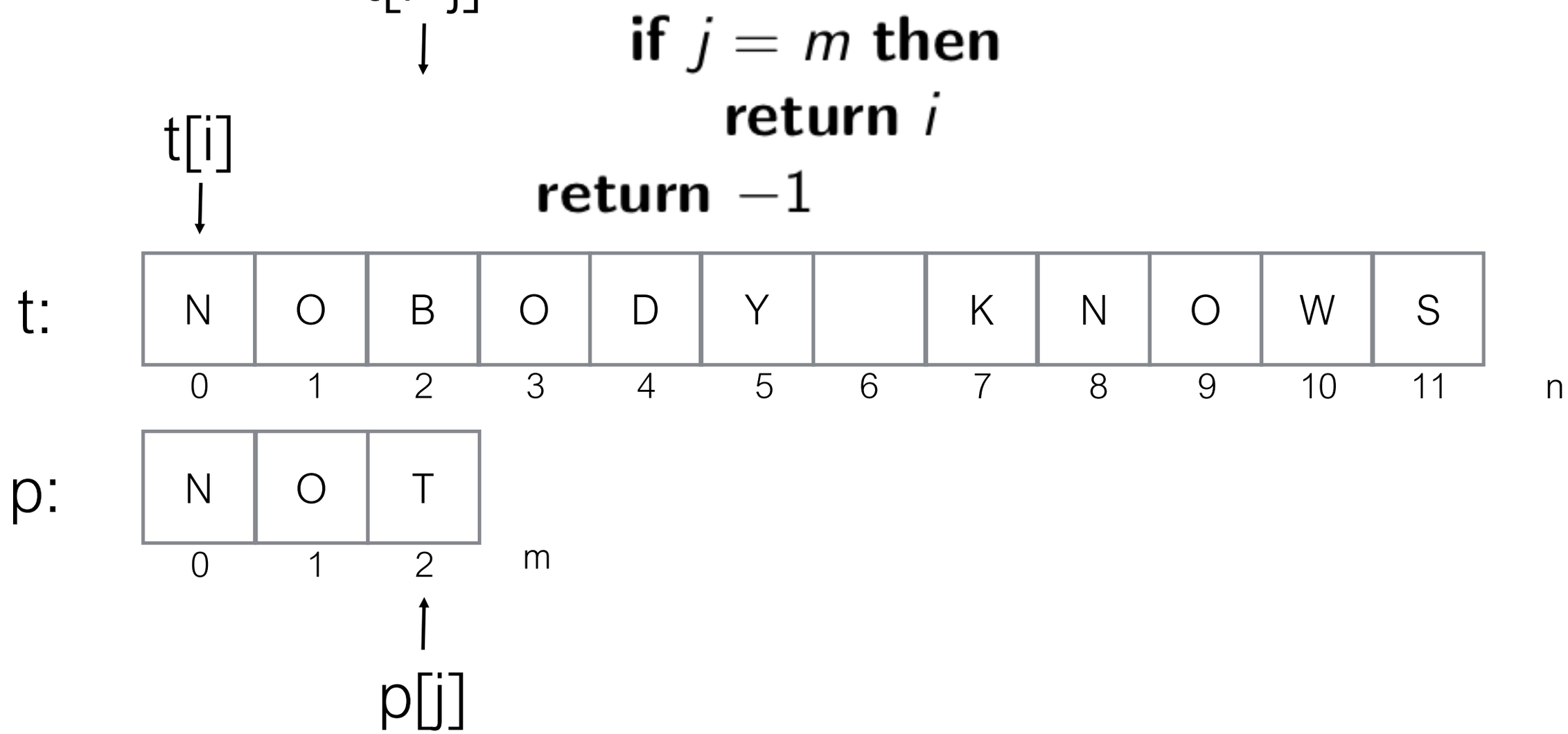
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Brute Force String Matching



THE UNIVERSITY OF MELBOURNE

```
for i ← 0 to n − m do
    j ← 0
    while j < m and p[j] = t[i + j] do
        j ← j + 1
    if j = m then
        return i
return −1
```



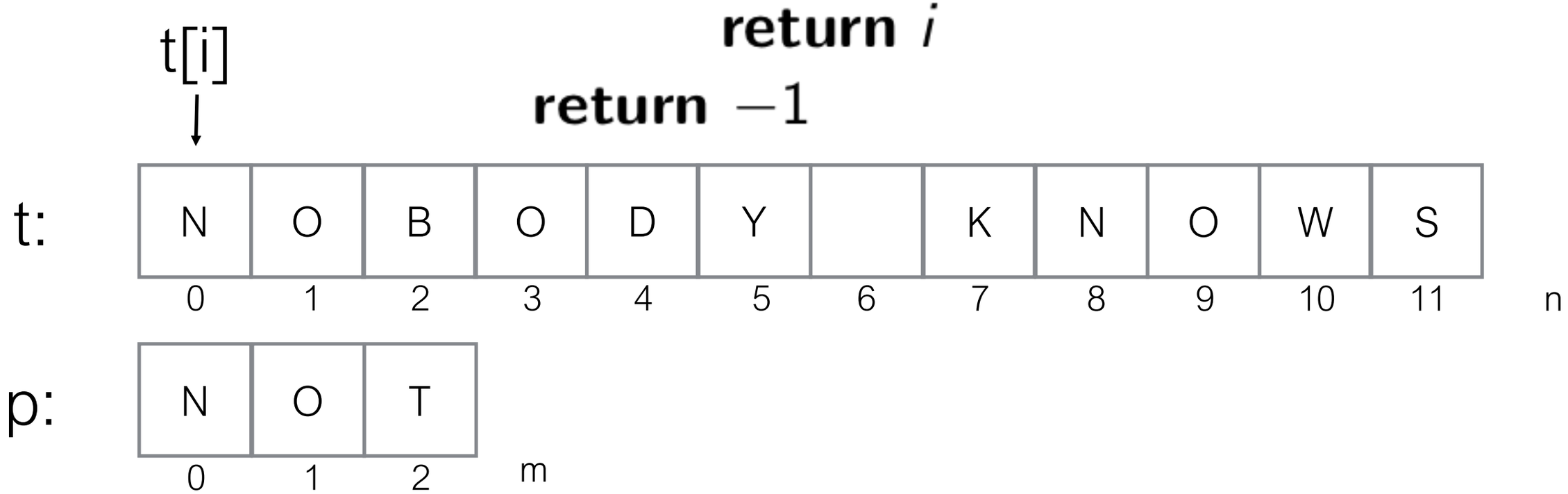
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Brute Force String Matching



THE UNIVERSITY OF MELBOURNE

```
for i ← 0 to n − m do
    j ← 0
    while j < m and p[j] = t[i + j] do
        j ← j + 1
    if j = m then
        return i
return −1
```



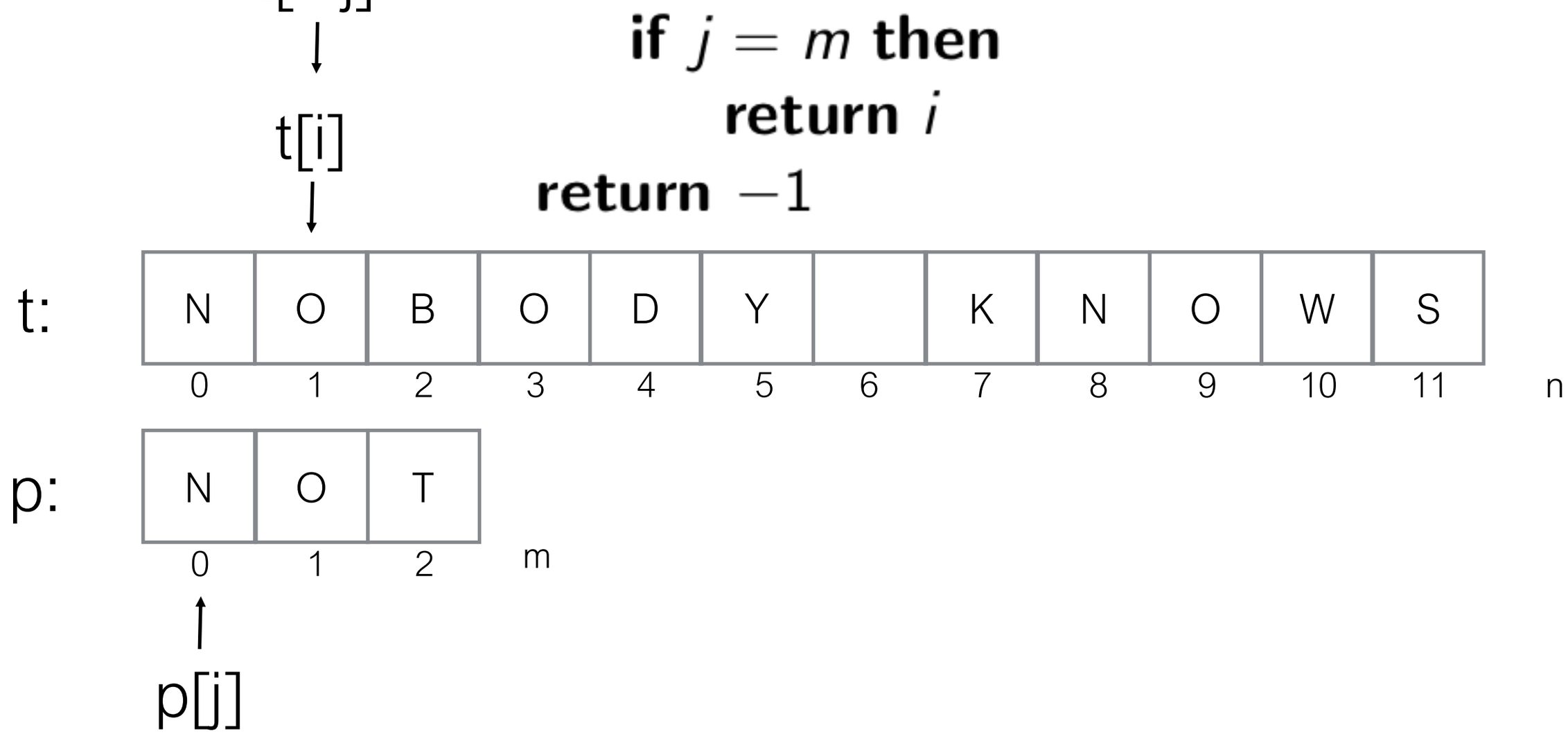
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Brute Force String Matching



THE UNIVERSITY OF MELBOURNE

```
for i ← 0 to n − m do
    j ← 0
    while j < m and p[j] = t[i + j] do
        j ← j + 1
    if j = m then
        return i
return −1
```



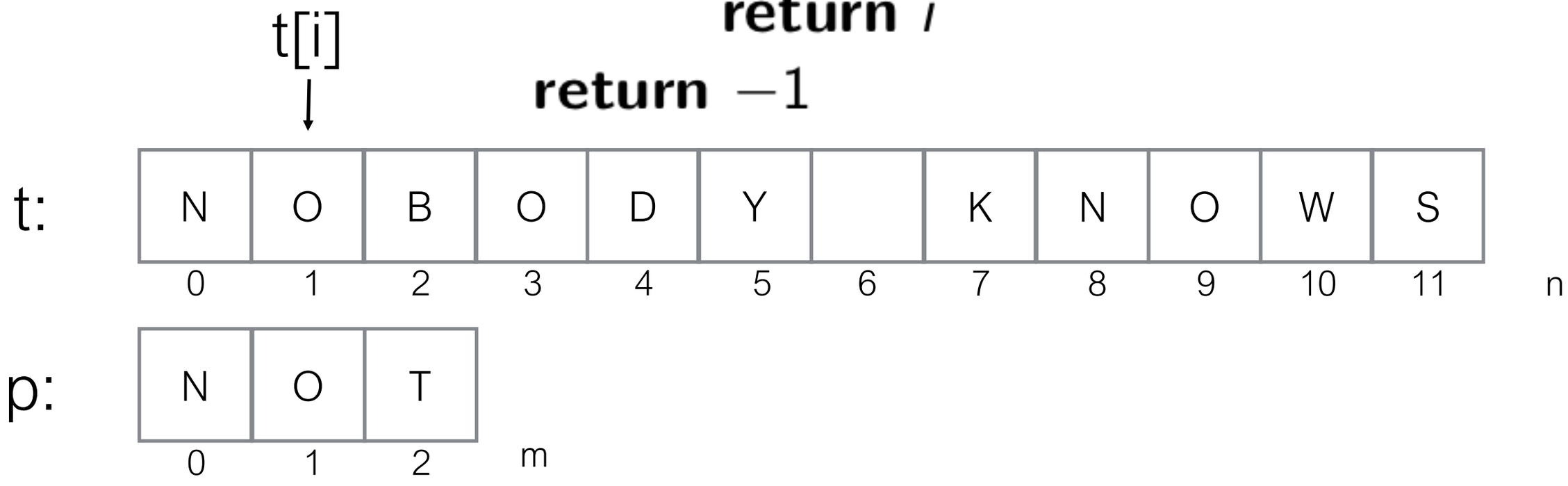
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Brute Force String Matching



THE UNIVERSITY OF MELBOURNE

```
for i ← 0 to n − m do
    j ← 0
    while j < m and p[j] = t[i + j] do
        j ← j + 1
    if j = m then
        return i
return −1
```



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Analysing Brute Force String Matching

Basic operation:

For each n - m + 1 positions in ***t***
we make up to m comparisons

Assuming n much larger than m: $O(mn)$ comparisons.

But for **random text** over reasonably large alphabet (e.g. English), **average** running time is linear in ***n***

There are better algorithms, for smaller alphabets such as binary strings or strings of DNA nucleobases. But for many purposes, the brute-force algorithm is acceptable.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Brute Force Geometric Algorithms: Closest Pair

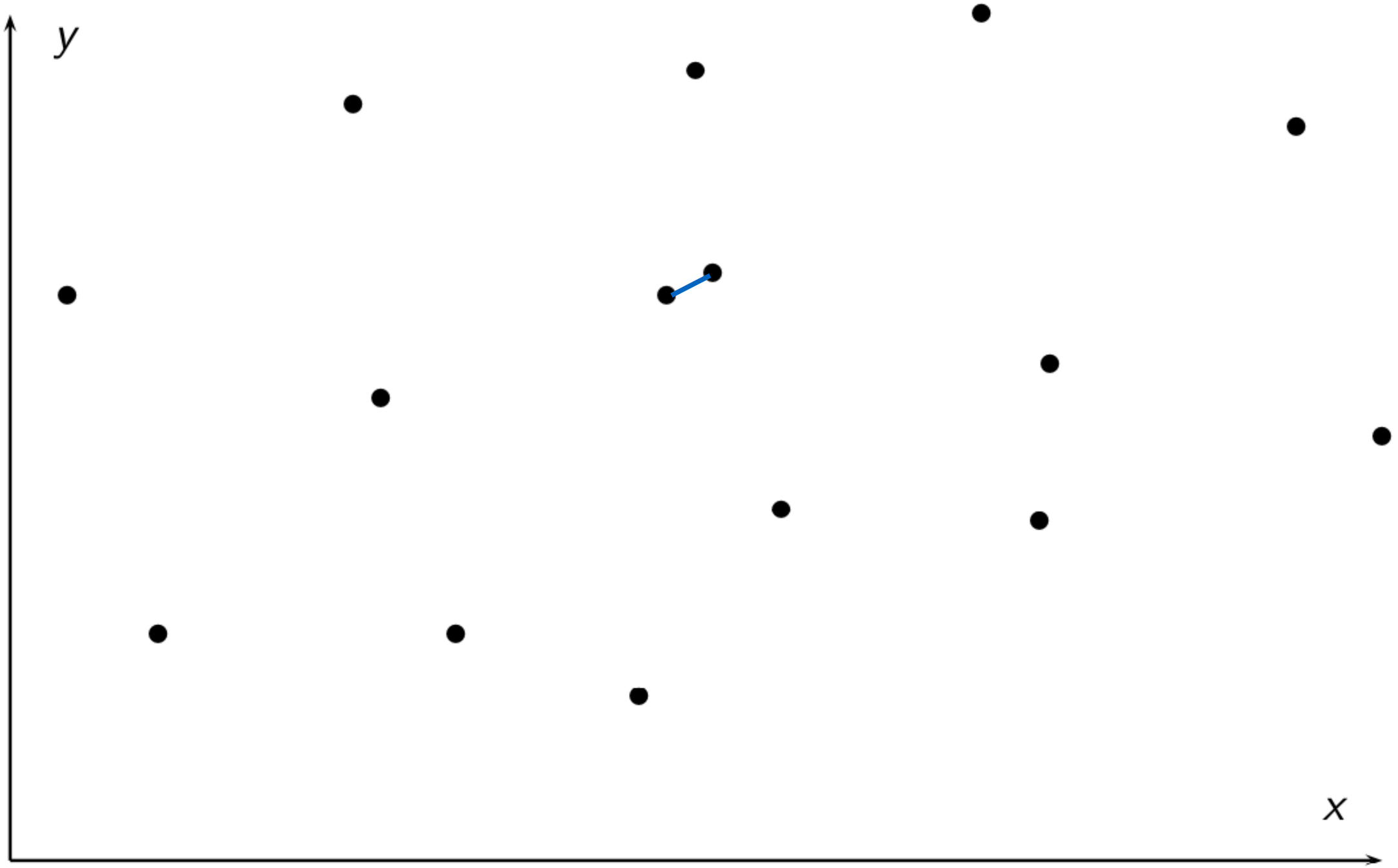
- **Problem:** Given n points in k-dimensional space, find a pair of points with minimal separating Euclidean distance.

- The brute force approach considers each pair in turn (except that once it has found the distance from x to y, it does not need to consider the distance from y to x).

- For simplicity, we look at the 2-dimensional case, the points being $(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})$.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Closest Pair Problem (2D)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

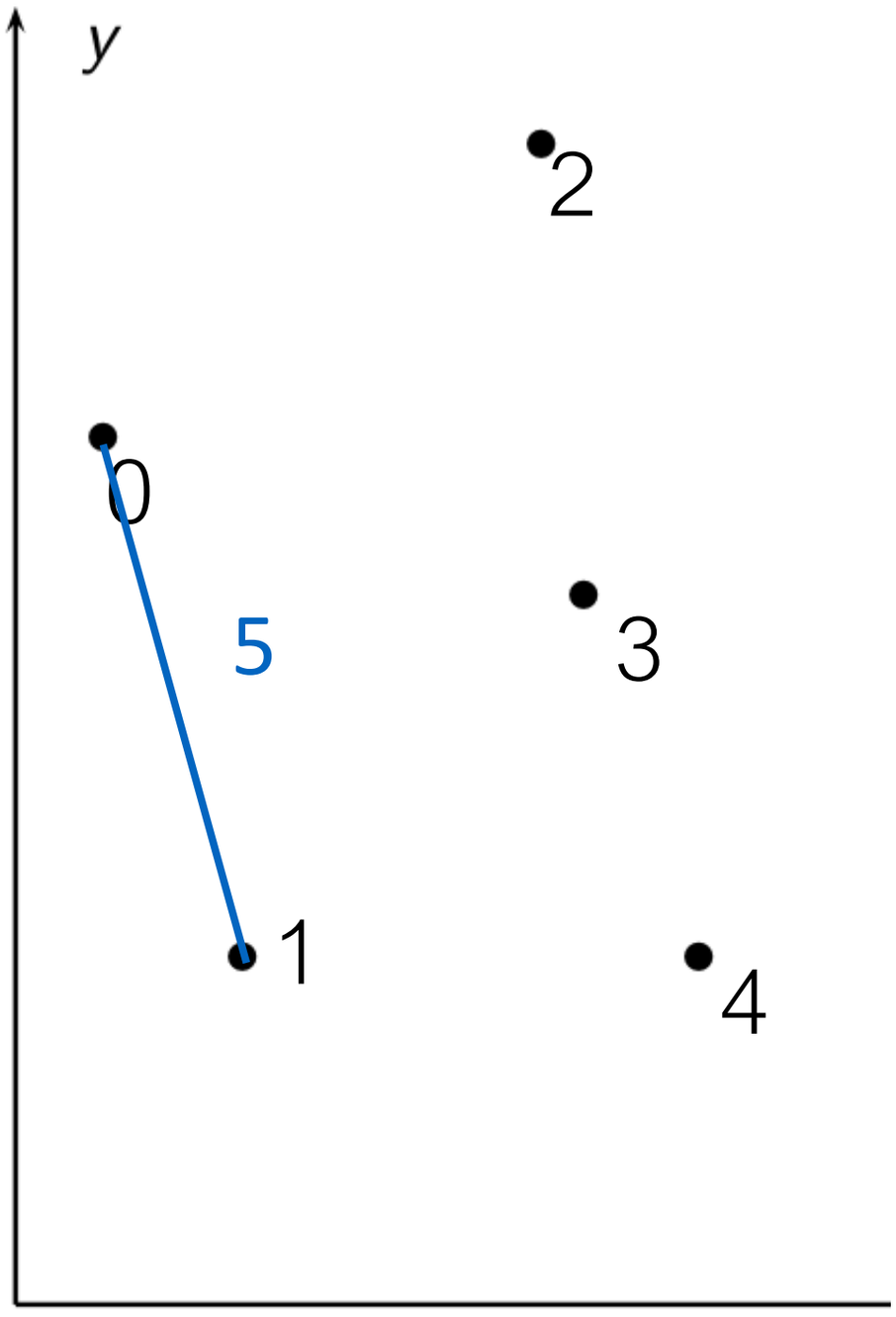
Brute Force Closest Pair

Try all combinations (x_i, y_i) and (x_j, y_j) with $i < j$:

$min \leftarrow \infty$
for $i \leftarrow 0$ to $n-2$ **do**
    for $j \leftarrow i+1$ to $n-1$ **do**
        $d \leftarrow sqrt((x_i - x_j)^2 + (y_i - y_j)^2)$ ▷ Distance for this pair
        if $d < min$ **then**
            $min \leftarrow d$ ▷ Smallest distance so far
            $p_1 \leftarrow i$ ▷ Remember this (i,j) combination
            $p_2 \leftarrow j$
return p_1, p_2

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

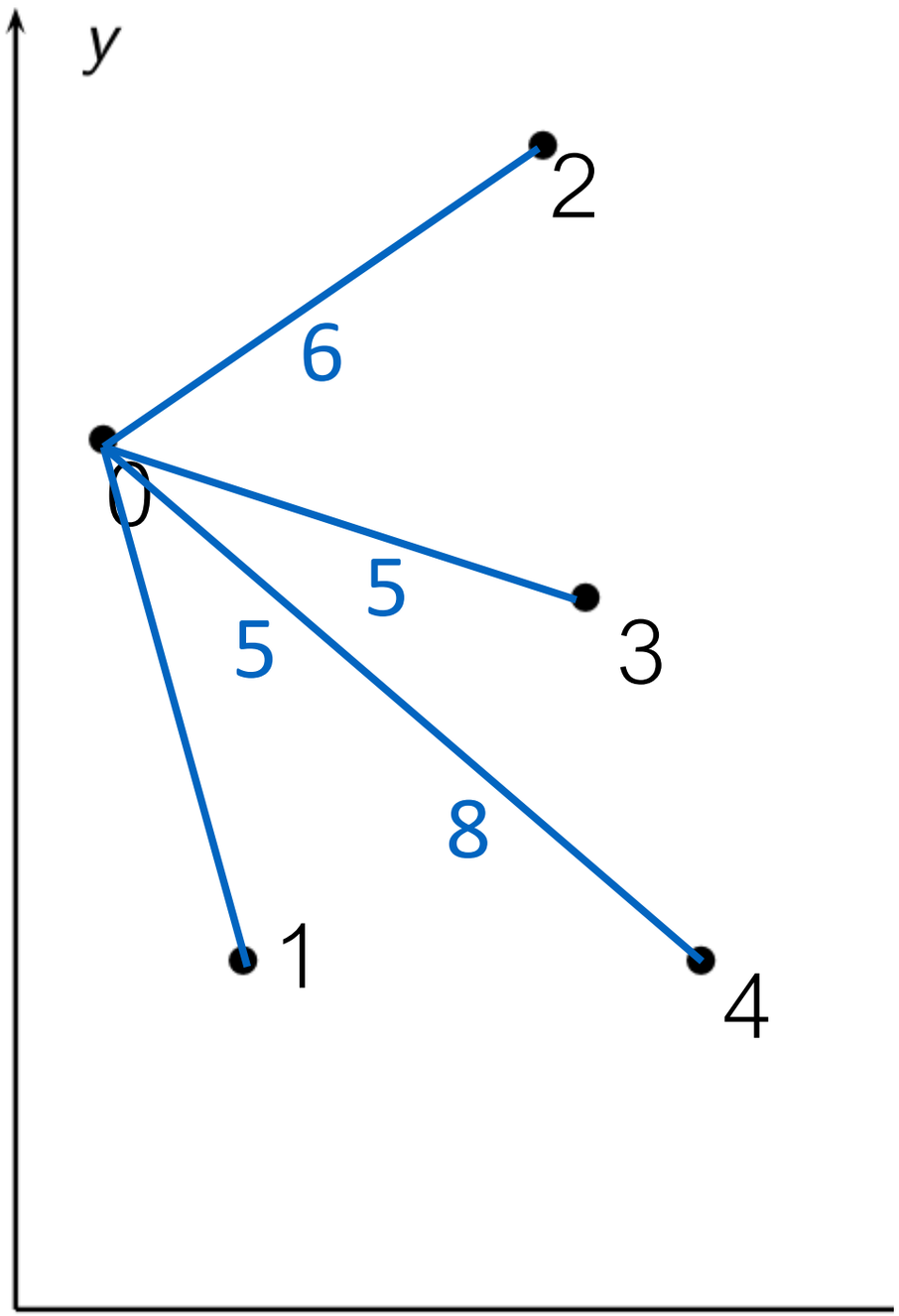
Closest Pair Problem (2D)



i=0 min=∞

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

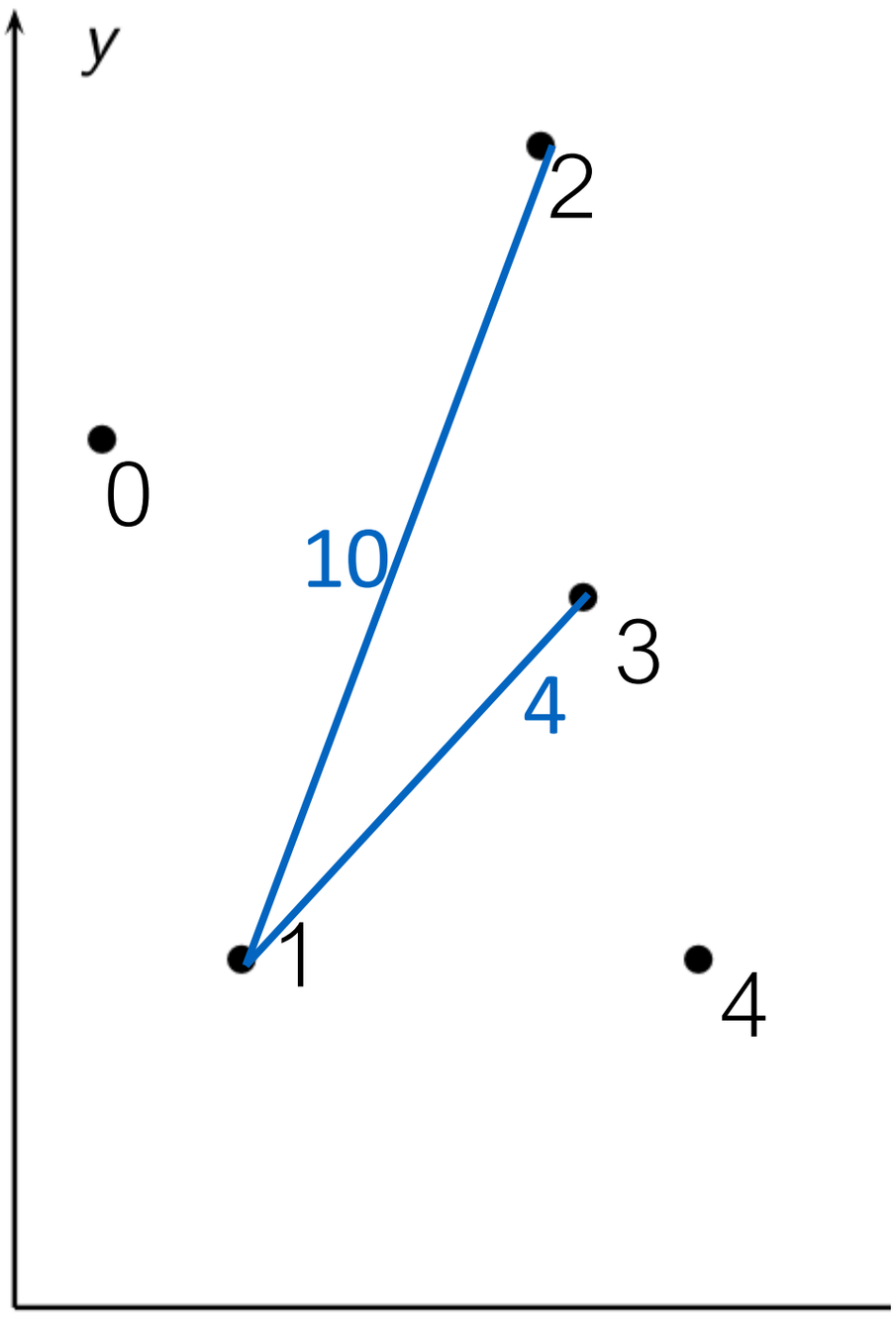
Closest Pair Problem (2D)



i=0 min=5

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

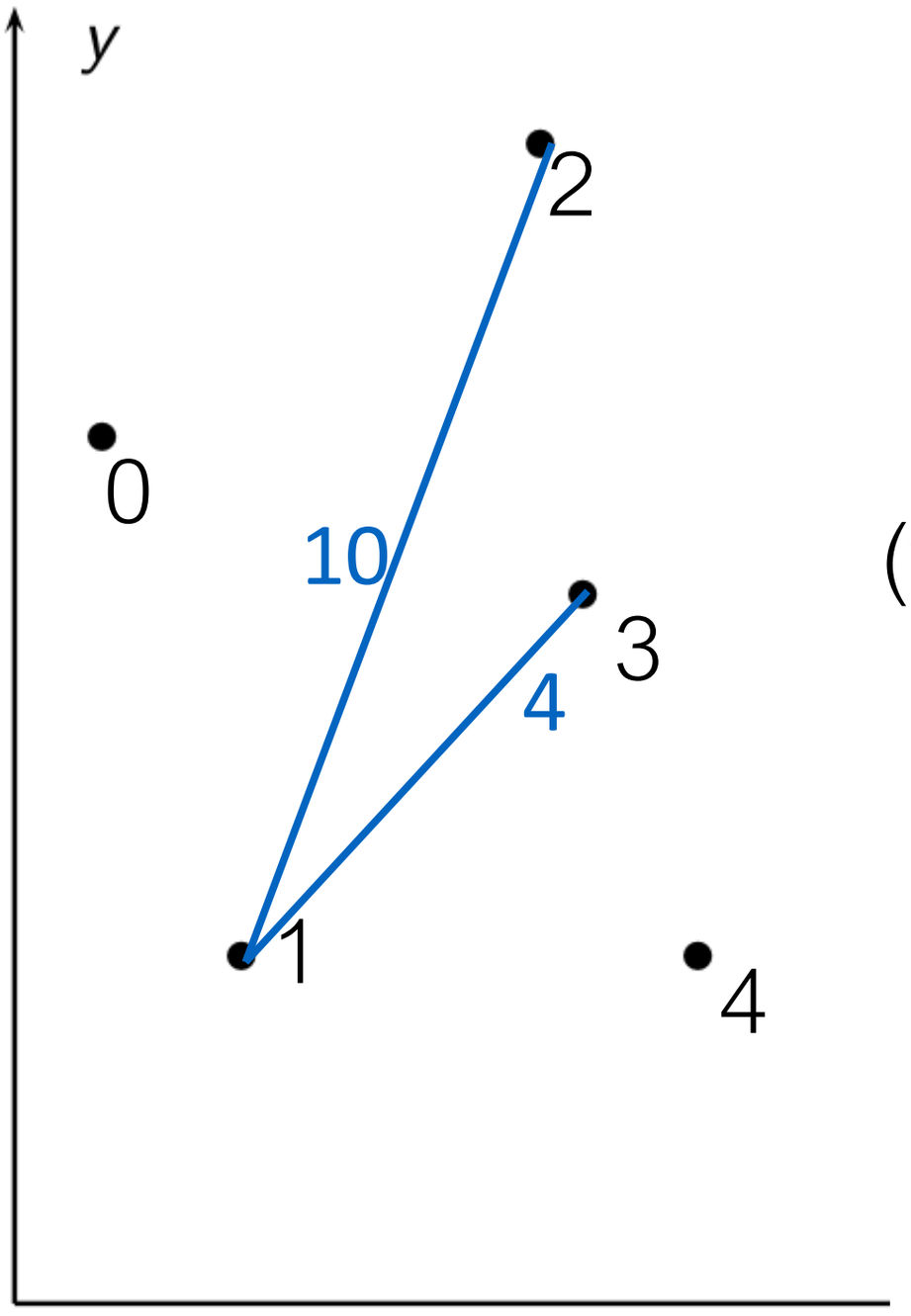
Closest Pair Problem (2D)



i=1 min=5

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Closest Pair Problem (2D)



i=1 min=4

and so on until
i=3 and j=4
(which will find the minimum here)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Analysing Brute Force Closest Pair Algorithm

- Not hard to see that the algorithm is $\Theta(n^2)$
- Note, however, that we can speed up the algorithm considerably, by utilising the monotonicity of the square root function $a < b \Longrightarrow \sqrt{a} < \sqrt{b}$
- Does that contradict the $\Theta(n^2)$ claim?
- Later we will see a clever divide and conquer approach leads to a $\Theta(n \log n)$ algorithm

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Brute Force Summary



THE UNIVERSITY OF MELBOURNE

- Simple, easy to program, widely applicable.
- Standard approach for small tasks.
- Reasonable algorithms for some problems.
- **But:** Generally inefficient—does not scale well.
- Use brute force for prototyping, or when it is known that input remains small.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Exhaustive Search

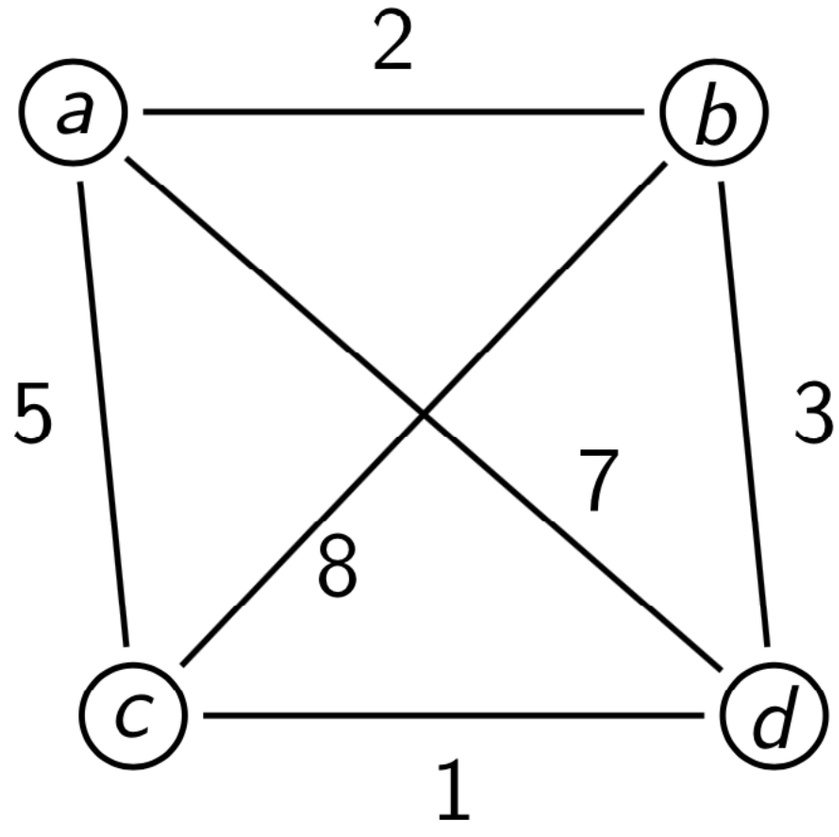
- Problem type:
  - Combinatorial decision or optimization problems
  - Search for an element with a particular property
  - Domain grows exponentially, for example all permutations

- The brute-force approach—generate and test:
  - Systematically construct all possible solutions
  - Evaluate each, keeping track of the best so far
  - When all potential solutions have been examined, return the best found

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

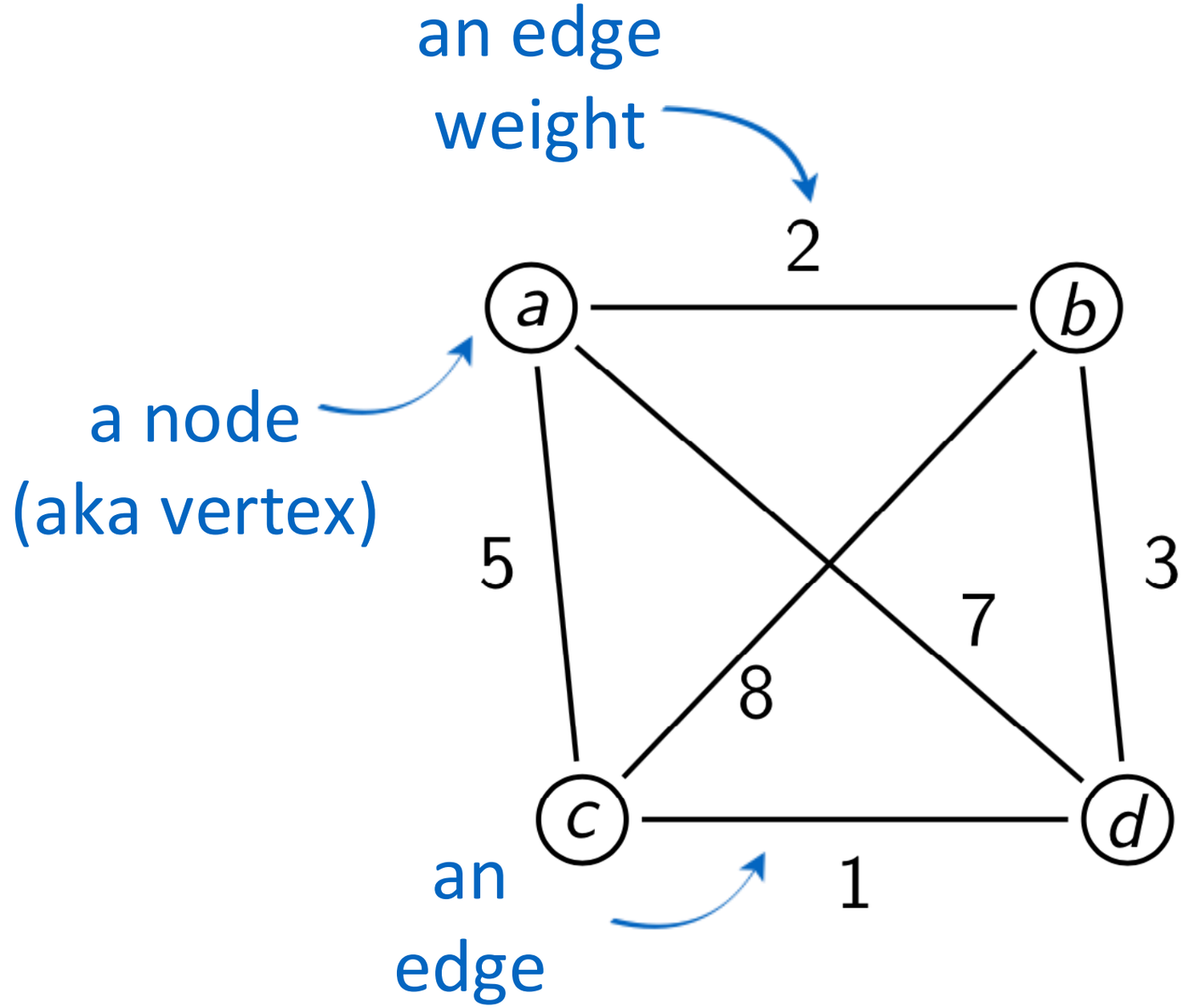
Example 1: Travelling Salesperson (TSP)

- Find the shortest **tour** (visiting each node exactly once before returning to the start) in a ***weighted, undirected*** graph.



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Side Note: Graph Concepts

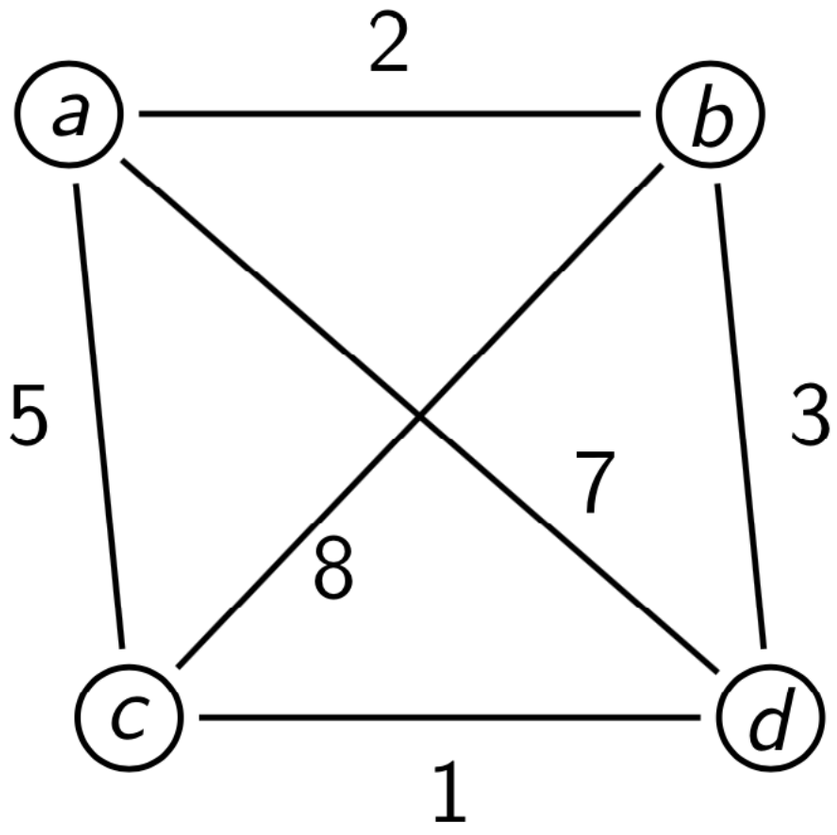


This graph is undirected since edges do not have directions associated with them.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example 1: Travelling Salesperson (TSP)

- Find the shortest **tour** (visiting each node exactly once before returning to the start) in a ***weighted, undirected*** graph.



Tours

$$
\begin{aligned}
a-b-c-d-a &: 18\\
a-b-d-c-a &: 11\\
a-c-b-d-a &: 23\\
a-c-d-b-a &: 11\\
a-d-b-c-a &: 23\\
a-d-c-b-a &: 18
\end{aligned}
$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

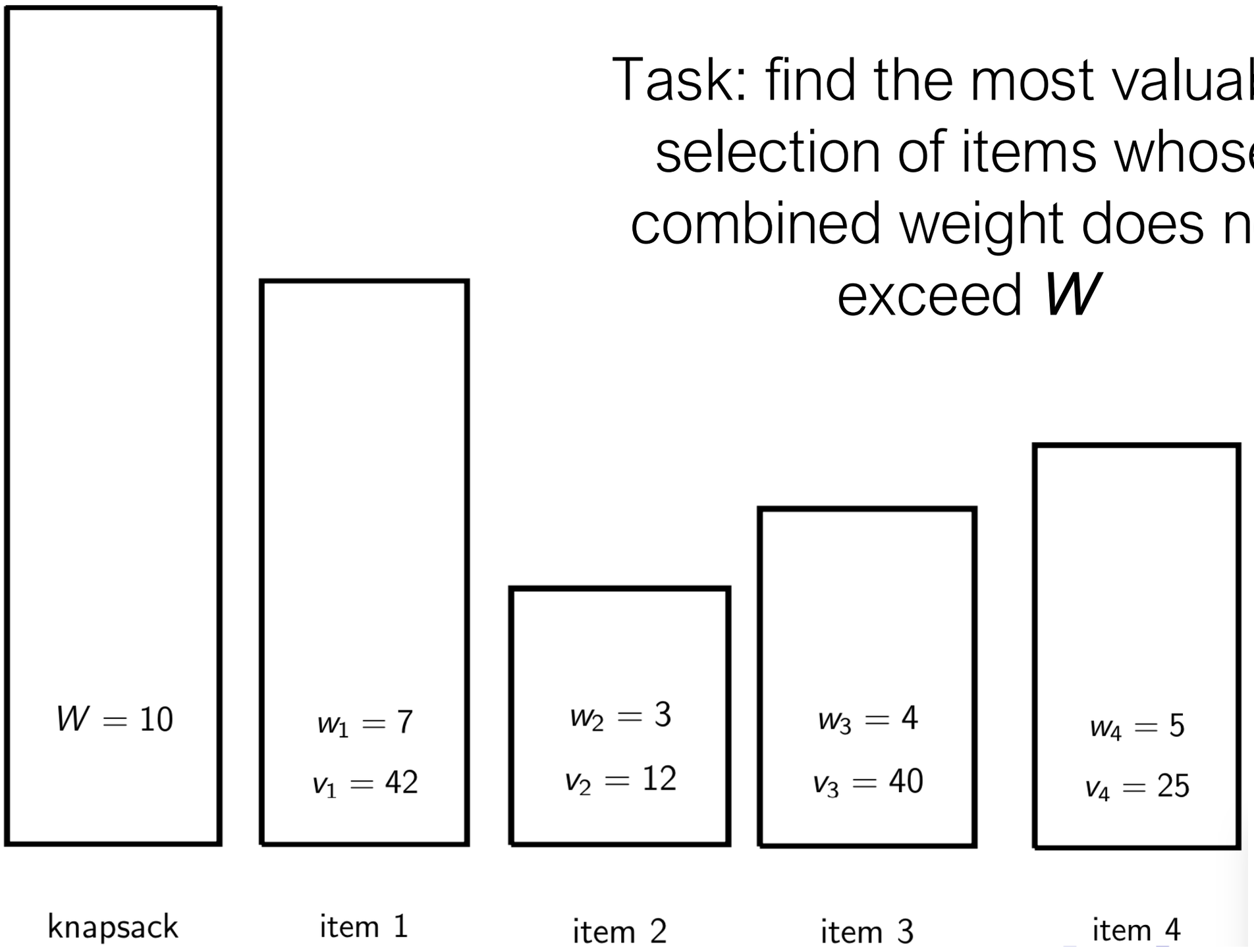
Example 2: Knapsack

- Given ***n*** items with
  - Weights: w_1, w_2, …, w_n
  - Values: v_1, v_2, …, v_n
  - Knapsack of capacity ***W***
- find the most valuable selection of items whose combined weight does not exceed ***W***



Copyright University of Melbourne 2026, provided under Creative C

Knapsack Example



Task: find the most valuable selection of items whose combined weight does not exceed W

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Knapsack: Example

Set	Weight	Value
$\emptyset$	0	0
$\{1\}$	7	42
$\{2\}$	3	12
$\{3\}$	4	40
$\{4\}$	5	25
$\{1,2\}$	10	54
$\{1,3\}$	11	NF
$\{1,4\}$	12	NF

Set	Weight	Value
$\{2,3\}$	7	52
$\{2,4\}$	8	37
$\{3,4\}$	9	65
$\{1,2,3\}$	14	NF
$\{1,2,4\}$	15	NF
$\{1,3,4\}$	16	NF
$\{2,3,4\}$	12	NF
$\{1,2,3,4\}$	19	NF

NF means “not feasible”: exhausts the capacity of the knapsack.

Later we’ll find a better algorithm based on
dynamic programming

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Comments on Exhaustive Search

- Exhaustive search algorithms have acceptable running times **only for very small instances.**

- In many cases there are better alternatives, for example, Eulerian tours, shortest paths, minimum spanning trees, . . .

- But for some problems, it is **known** that there is essentially no better alternative.

- For a large class of important problems, it **appears** that there is no better alternative, but we have no proof either way.

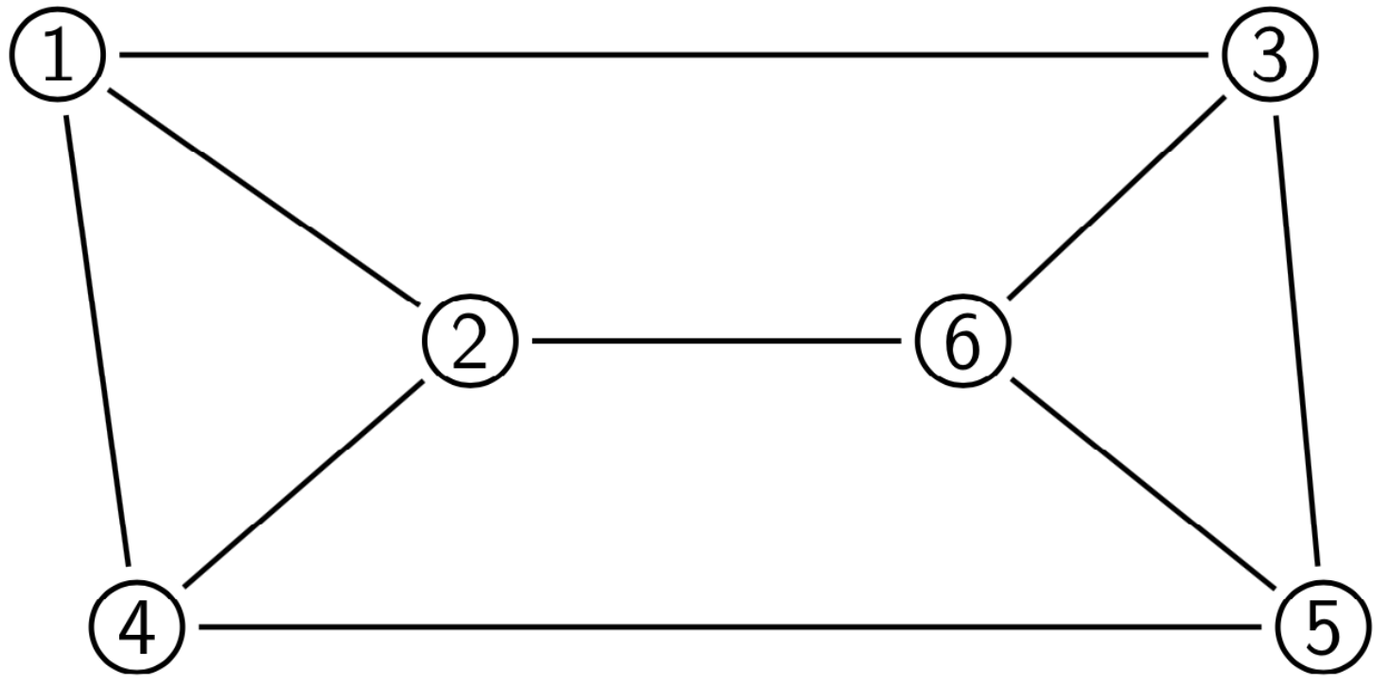
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hamiltonian Tours

- The Hamiltonian tour problem is this:

- In a given undirected graph, is there a simple tour (a path that visits each **node** exactly once, except it returns to the starting node)?

- Is there a Hamiltonian tour of this graph?



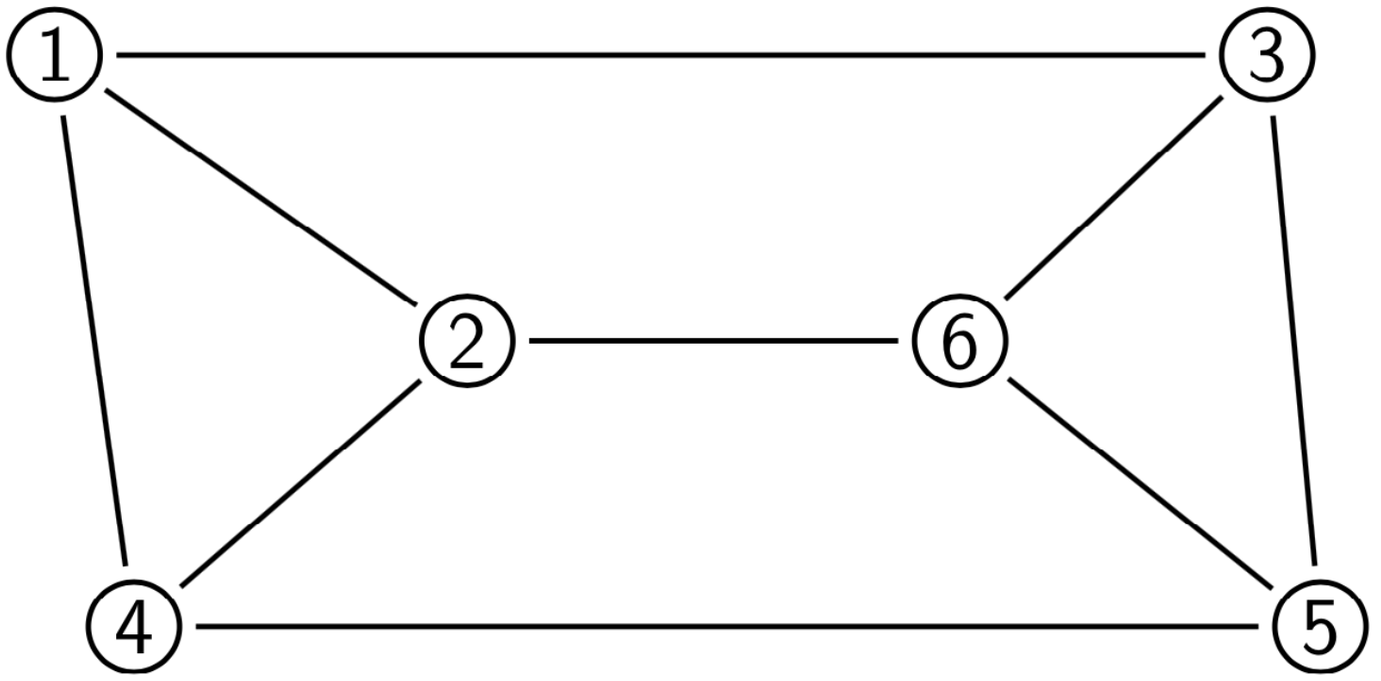
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Eulerian Tours

- The Eulerian tour problem is this:

- In a given undirected graph, is there a path which visits each **edge** exactly once?

- Is there a Eulerian tour of this graph?



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Hard and Easy Problems

- Recall that by a **problem** we usually mean a parametric problem: an infinite family of problem "instances".

- Sometimes our intuition about the difficulty of problems is not very reliable. The Hamiltonian Tour problem and the Eulerian Tour problem look very similar, but one is hard and the other is easy. We shall see more examples of this phenomenon later.

- For many important **optimization** problems we do not know of solutions that are essentially better than exhaustive search (a whole raft of **NP-complete** problems, including TSP, knapsack).

- In those cases we may look for **approximation algorithms** that are fast and still find solutions that are reasonably close to the optimal.

- We plan to return to this idea in Week 12.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Next Up

- **Recursion** as a problem-solving technique

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License